Transfer Learning using PyTorch

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!nvidia-smi
```

```
Fri Mar  6 09:38:51 2026
+-----------------------------------------------------------------------------
--------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07
CUDA Version: 13.0     |
+-----------------------------------------+------------------------
+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A |
Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage |
GPU-Util  Compute M. |
|                                         |                        |
MIG M. |
|
=========================================+========================+===
===================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |
0 |
| N/A   68C    P8             11W /   70W |       0MiB /  15360MiB |
0%      Default |
|                                         |                        |
N/A |
+-----------------------------------------+------------------------
+----------------------+

+-----------------------------------------------------------------------------
--------------------+
| Processes:
|
|  GPU   GI   CI              PID   Type   Process name
GPU Memory |
|        ID   ID
Usage      |
|
=============================================================================
===================|
|  No running processes found
|
```

```
+-----------------------------------------------------------------------
--------------------+
```

Load Important Libraries

```
# License: BSD
# Author: Sasank Chilamkurthy

import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import lr_scheduler
import torch.backends.cudnn as cudnn
import numpy as np
import torchvision
from torchvision import datasets, models, transforms
import matplotlib.pyplot as plt
import time
import os
from PIL import Image
from tempfile import TemporaryDirectory

cudnn.benchmark = True
plt.ion()   # interactive mode
```

```
<contextlib.ExitStack at 0x7e70e16c3530>
```

Load Data

We will use torchvision and torch.utils.data packages for loading the data.

The problem we're going to solve today is to train a model to classify **ants** and **bees**. We have about 120 training images each for ants and bees. There are 75 validation images for each class. Usually, this is a very small dataset to generalize upon, if trained from scratch. Since we are using transfer learning, we should be able to generalize reasonably well.

This dataset is a very small subset of imagenet.

```
!rm -R /content/hymenoptera_data/train/.ipynb_checkpoints
!ls /content/hymenoptera_data/test/train -a   #to make sure that the
deletion has occurred

!rm -R /content/hymenoptera_data/val/.ipynb_checkpoints
!ls /content/hymenoptera_data/val -a   #to make sure that the deletion
has occurred
```

```
rm: cannot remove
'/content/hymenoptera_data/train/.ipynb_checkpoints': No such file or
```

```
directory
ls: cannot access '/content/hymenoptera_data/test/train': No such file
or directory
rm: cannot remove '/content/hymenoptera_data/val/.ipynb_checkpoints':
No such file or directory
ls: cannot access '/content/hymenoptera_data/val': No such file or
directory
```

```
# Data augmentation and normalization for training
# Just normalization for validation
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])
    ]),
}

data_dir = '/content/drive/MyDrive/CPE 313 - Advanced Machine Learning
and Deep Learning/Hands-on Activity 4.2 Transfer Learning on
PyTorch/hymenoptera_data'
image_datasets = {x: datasets.ImageFolder(os.path.join(data_dir, x),
                                          data_transforms[x])
                  for x in ['train', 'val']}
dataloaders = {x: torch.utils.data.DataLoader(image_datasets[x],
batch_size=4,
                                              shuffle=True,
num_workers=4)
               for x in ['train', 'val']}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
class_names = image_datasets['train'].classes

# We want to be able to train our model on an `accelerator
<https://pytorch.org/docs/stable/torch.html#accelerators>`__
# such as CUDA, MPS, MTIA, or XPU. If the current accelerator is
available, we will use it. Otherwise, we use the CPU.

device = torch.accelerator.current_accelerator().type if
torch.accelerator.is_available() else "cpu"
print(f"Using {device} device")
```

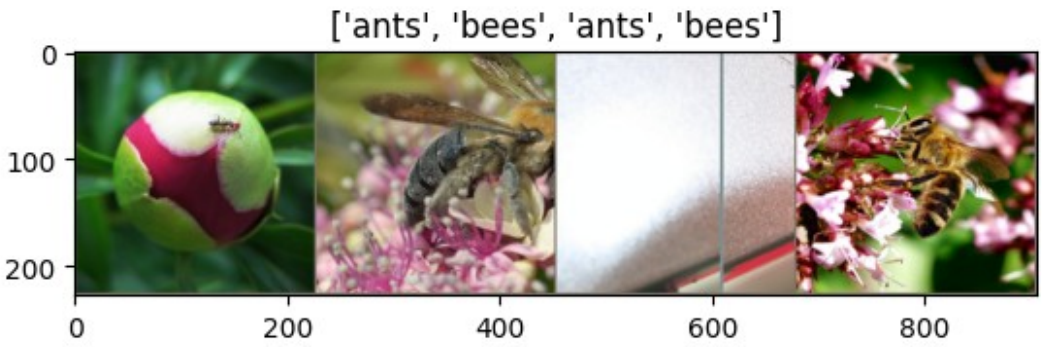
```
Using cuda device

/usr/local/lib/python3.12/dist-packages/torch/utils/data/
dataloader.py:424: UserWarning: This DataLoader will create 4 worker
processes in total. Our suggested max number of worker in current
system is 2, which is smaller than what this DataLoader is going to
create. Please be aware that excessive worker creation might get
DataLoader running slow or even freeze, lower the worker number to
avoid potential slowness/freeze if necessary.
  self.check_worker_number_rationality()
```

Visualize a few images

Let's visualize a few training images so as to understand the data augmentations.

```
def imshow(inp, title=None):
    """Display image for Tensor."""
    inp = inp.numpy().transpose((1, 2, 0))
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    inp = std * inp + mean
    inp = np.clip(inp, 0, 1)
    plt.imshow(inp)
    if title is not None:
        plt.title(title)
    plt.pause(0.001)  # pause a bit so that plots are updated


# Get a batch of training data
inputs, classes = next(iter(dataloaders['train']))

# Make a grid from batch
out = torchvision.utils.make_grid(inputs)

imshow(out, title=[class_names[x] for x in classes])
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/
dataloader.py:432: UserWarning: This DataLoader will create 4 worker
processes in total. Our suggested max number of worker in current
system is 2, which is smaller than what this DataLoader is going to
create. Please be aware that excessive worker creation might get
DataLoader running slow or even freeze, lower the worker number to
avoid potential slowness/freeze if necessary.
  self.check_worker_number_rationality()
```



Training the model

Now, let's write a general function to train a model. Here, we will illustrate:

- Scheduling the learning rate
- Saving the best model

In the following, parameter `scheduler` is an LR scheduler object from `torch.optim.lr_scheduler`.

```
def train_model(model, criterion, optimizer, scheduler,
num_epochs=25):
    since = time.time()

    # Create a temporary directory to save training checkpoints
    with TemporaryDirectory() as tempdir:
        best_model_params_path = os.path.join(tempdir,
'best_model_params.pt')

        torch.save(model.state_dict(), best_model_params_path)
        best_acc = 0.0

        for epoch in range(num_epochs):
            print(f'Epoch {epoch}/{num_epochs - 1}')
            print('-' * 10)

            # Each epoch has a training and validation phase
            for phase in ['train', 'val']:
                if phase == 'train':
                    model.train()  # Set model to training mode
                else:
                    model.eval()   # Set model to evaluate mode

                running_loss = 0.0
                running_corrects = 0

                # Iterate over data.
```

```
                for inputs, labels in dataloaders[phase]:
                    inputs = inputs.to(device)
                    labels = labels.to(device)

                    # zero the parameter gradients
                    optimizer.zero_grad()

                    # forward
                    # track history if only in train
                    with torch.set_grad_enabled(phase == 'train'):
                        outputs = model(inputs)
                        _, preds = torch.max(outputs, 1)
                        loss = criterion(outputs, labels)

                        # backward + optimize only if in training
phase
                        if phase == 'train':
                            loss.backward()
                            optimizer.step()

                    # statistics
                    running_loss += loss.item() * inputs.size(0)
                    running_corrects += torch.sum(preds ==
labels.data)
                if phase == 'train':
                    scheduler.step()

                epoch_loss = running_loss / dataset_sizes[phase]
                epoch_acc = running_corrects.double() /
dataset_sizes[phase]

                print(f'{phase} Loss: {epoch_loss:.4f} Acc:
{epoch_acc:.4f}')

                # deep copy the model
                if phase == 'val' and epoch_acc > best_acc:
                    best_acc = epoch_acc
                    torch.save(model.state_dict(),
best_model_params_path)

            print()

        time_elapsed = time.time() - since
        print(f'Training complete in {time_elapsed // 60:.0f}m
{time_elapsed % 60:.0f}s')
        print(f'Best val Acc: {best_acc:4f}')

        # load best model weights
        model.load_state_dict(torch.load(best_model_params_path,
```

```
weights_only=True))
    return model
```

Visualizing the model predictions

Generic function to display predictions for a few images

```
def visualize_model(model, num_images=6):
    was_training = model.training
    model.eval()
    images_so_far = 0
    fig = plt.figure()

    with torch.no_grad():
        for i, (inputs, labels) in enumerate(dataloaders['val']):
            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)

            for j in range(inputs.size()[0]):
                images_so_far += 1
                ax = plt.subplot(num_images//2, 2, images_so_far)
                ax.axis('off')
                ax.set_title(f'predicted: {class_names[preds[j]]}')
                imshow(inputs.cpu().data[j])

                if images_so_far == num_images:
                    model.train(mode=was_training)
                    return
        model.train(mode=was_training)
```

Finetuning the ConvNet

Load a pretrained model and reset final fully connected layer.

```
model_ft = models.resnet18(weights='IMAGENET1K_V1')
num_ftrs = model_ft.fc.in_features
# Here the size of each output sample is set to 2.
# Alternatively, it can be generalized to ``nn.Linear(num_ftrs,
len(class_names))``.
model_ft.fc = nn.Linear(num_ftrs, 2)

model_ft = model_ft.to(device)
```

```
criterion = nn.CrossEntropyLoss()

# Observe that all parameters are being optimized
optimizer_ft = optim.SGD(model_ft.parameters(), lr=0.001,
momentum=0.9)

# Decay LR by a factor of 0.1 every 7 epochs
exp_lr_scheduler = lr_scheduler.StepLR(optimizer_ft, step_size=7,
gamma=0.1)
```

```
Downloading: "https://download.pytorch.org/models/resnet18-
f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-
f37072fd.pth

100%|██████████| 44.7M/44.7M [00:00<00:00, 208MB/s]
```

Train and evaluate

It should take around 15-25 min on CPU. On GPU though, it takes less than a minute.

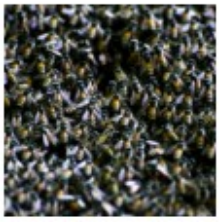
```
model_ft = train_model(model_ft, criterion, optimizer_ft,
exp_lr_scheduler,
                       num_epochs=25)
```

```
Epoch 0/24
----------
train Loss: 0.6570 Acc: 0.6926
val Loss: 0.3359 Acc: 0.8497

Epoch 1/24
----------
train Loss: 0.6116 Acc: 0.7746
val Loss: 0.4281 Acc: 0.8366

Epoch 2/24
----------
train Loss: 0.4923 Acc: 0.7992
val Loss: 0.4223 Acc: 0.8497

Epoch 3/24
----------
train Loss: 0.5877 Acc: 0.7910
val Loss: 0.5362 Acc: 0.8627

Epoch 4/24
----------
train Loss: 0.4831 Acc: 0.7910
```

```
val Loss: 0.3766 Acc: 0.8758

Epoch 5/24
----------
train Loss: 0.5262 Acc: 0.7828
val Loss: 0.3305 Acc: 0.8627

Epoch 6/24
----------
train Loss: 0.4504 Acc: 0.8443
val Loss: 0.4016 Acc: 0.8889

Epoch 7/24
----------
train Loss: 0.4174 Acc: 0.8156
val Loss: 0.3805 Acc: 0.8889

Epoch 8/24
----------
train Loss: 0.4960 Acc: 0.7787
val Loss: 0.3801 Acc: 0.8889

Epoch 9/24
----------
train Loss: 0.3409 Acc: 0.8443
val Loss: 0.2993 Acc: 0.8954

Epoch 10/24
----------
train Loss: 0.3908 Acc: 0.8402
val Loss: 0.2948 Acc: 0.9020

Epoch 11/24
----------
train Loss: 0.3716 Acc: 0.8443
val Loss: 0.2773 Acc: 0.9085

Epoch 12/24
----------
train Loss: 0.2946 Acc: 0.8648
val Loss: 0.3120 Acc: 0.9085

Epoch 13/24
----------
train Loss: 0.2813 Acc: 0.8648
val Loss: 0.2762 Acc: 0.9150

Epoch 14/24
----------
train Loss: 0.3258 Acc: 0.8648
```

```
val Loss: 0.2890 Acc: 0.9216

Epoch 15/24
----------
train Loss: 0.2858 Acc: 0.8566
val Loss: 0.2910 Acc: 0.9216

Epoch 16/24
----------
train Loss: 0.2060 Acc: 0.8975
val Loss: 0.2924 Acc: 0.9085

Epoch 17/24
----------
train Loss: 0.3029 Acc: 0.8852
val Loss: 0.2884 Acc: 0.9150

Epoch 18/24
----------
train Loss: 0.2680 Acc: 0.8770
val Loss: 0.2824 Acc: 0.9216

Epoch 19/24
----------
train Loss: 0.3084 Acc: 0.8402
val Loss: 0.2662 Acc: 0.9216

Epoch 20/24
----------
train Loss: 0.3270 Acc: 0.8607
val Loss: 0.2863 Acc: 0.9216

Epoch 21/24
----------
train Loss: 0.3384 Acc: 0.8361
val Loss: 0.3214 Acc: 0.9085

Epoch 22/24
----------
train Loss: 0.2081 Acc: 0.9098
val Loss: 0.2722 Acc: 0.9150

Epoch 23/24
----------
train Loss: 0.1972 Acc: 0.9139
val Loss: 0.2709 Acc: 0.9085

Epoch 24/24
----------
train Loss: 0.2631 Acc: 0.8730
```

```
val Loss: 0.2989 Acc: 0.9085

Training complete in 2m 6s
Best val Acc: 0.921569

visualize_model(model_ft)
```



predicted: ants

predicted: ants

ConvNet as fixed feature extractor

Here, we need to freeze all the network except the final layer. We need to set `requires_grad = False` to freeze the parameters so that the gradients are not computed in `backward()`.

You can read more about this in the documentation here.

```
model_conv = torchvision.models.resnet18(weights='IMAGENET1K_V1')
for param in model_conv.parameters():
    param.requires_grad = False

# Parameters of newly constructed modules have requires_grad=True by
default
num_ftrs = model_conv.fc.in_features
model_conv.fc = nn.Linear(num_ftrs, 2)

model_conv = model_conv.to(device)

criterion = nn.CrossEntropyLoss()

# Observe that only parameters of final layer are being optimized as
# opposed to before.
optimizer_conv = optim.SGD(model_conv.fc.parameters(), lr=0.001,
momentum=0.9)

# Decay LR by a factor of 0.1 every 7 epochs
exp_lr_scheduler = lr_scheduler.StepLR(optimizer_conv, step_size=7,
gamma=0.1)
```

Train and evaluate

On CPU this will take about half the time compared to previous scenario. This is expected as gradients don't need to be computed for most of the network. However, forward does need to be computed.

```
model_conv = train_model(model_conv, criterion, optimizer_conv,
                         exp_lr_scheduler, num_epochs=25)
```

```
Epoch 0/24
----------
train Loss: 0.6388 Acc: 0.6557
val Loss: 0.1973 Acc: 0.9281

Epoch 1/24
----------
train Loss: 0.5320 Acc: 0.7664
val Loss: 0.3348 Acc: 0.8693

Epoch 2/24
----------
train Loss: 0.4607 Acc: 0.7828
val Loss: 0.2521 Acc: 0.9085

Epoch 3/24
----------
train Loss: 0.5298 Acc: 0.7664
val Loss: 0.1756 Acc: 0.9281

Epoch 4/24
----------
train Loss: 0.5983 Acc: 0.7336
val Loss: 0.3949 Acc: 0.8366

Epoch 5/24
----------
train Loss: 0.5269 Acc: 0.7787
val Loss: 0.2700 Acc: 0.9150

Epoch 6/24
----------
train Loss: 0.4898 Acc: 0.8115
val Loss: 0.1812 Acc: 0.9346

Epoch 7/24
----------
train Loss: 0.3310 Acc: 0.8402
val Loss: 0.1792 Acc: 0.9412

Epoch 8/24
```

```
----------
train Loss: 0.3921 Acc: 0.8361
val Loss: 0.1875 Acc: 0.9412

Epoch 9/24
----------
train Loss: 0.2654 Acc: 0.8852
val Loss: 0.1901 Acc: 0.9346

Epoch 10/24
----------
train Loss: 0.3676 Acc: 0.8443
val Loss: 0.1691 Acc: 0.9542

Epoch 11/24
----------
train Loss: 0.3884 Acc: 0.8238
val Loss: 0.2006 Acc: 0.9477

Epoch 12/24
----------
train Loss: 0.4111 Acc: 0.8402
val Loss: 0.1766 Acc: 0.9477

Epoch 13/24
----------
train Loss: 0.4375 Acc: 0.8197
val Loss: 0.1964 Acc: 0.9346

Epoch 14/24
----------
train Loss: 0.3298 Acc: 0.8566
val Loss: 0.2040 Acc: 0.9281

Epoch 15/24
----------
train Loss: 0.3130 Acc: 0.8443
val Loss: 0.2092 Acc: 0.9412

Epoch 16/24
----------
train Loss: 0.3522 Acc: 0.8279
val Loss: 0.1889 Acc: 0.9412

Epoch 17/24
----------
train Loss: 0.2701 Acc: 0.8811
val Loss: 0.1913 Acc: 0.9346

Epoch 18/24
```

```
----------
train Loss: 0.3235 Acc: 0.8484
val Loss: 0.1765 Acc: 0.9477

Epoch 19/24
----------
train Loss: 0.3729 Acc: 0.8320
val Loss: 0.1640 Acc: 0.9477

Epoch 20/24
----------
train Loss: 0.4172 Acc: 0.8115
val Loss: 0.1941 Acc: 0.9412

Epoch 21/24
----------
train Loss: 0.3108 Acc: 0.8566
val Loss: 0.2152 Acc: 0.9412

Epoch 22/24
----------
train Loss: 0.3894 Acc: 0.8361
val Loss: 0.1848 Acc: 0.9412

Epoch 23/24
----------
train Loss: 0.3235 Acc: 0.8648
val Loss: 0.1987 Acc: 0.9412

Epoch 24/24
----------
train Loss: 0.3678 Acc: 0.8320
val Loss: 0.1751 Acc: 0.9477

Training complete in 1m 36s
Best val Acc: 0.954248

visualize_model(model_conv)

plt.ioff()
plt.show()
```

predicted: ants

predicted: ants

predicted: bees

predicted: bees

predicted: bees

Inference on custom images

Use the trained model to make predictions on custom images and visualize the predicted class labels along with the images.

```
def visualize_model_predictions(model,img_path):
    was_training = model.training
    model.eval()

    img = Image.open(img_path)
    img = data_transforms['val'](img)
    img = img.unsqueeze(0)
    img = img.to(device)

    with torch.no_grad():
        outputs = model(img)
        _, preds = torch.max(outputs, 1)

        ax = plt.subplot(2,2,1)
        ax.axis('off')
        ax.set_title(f'Predicted: {class_names[preds[0]]}')
        imshow(img.cpu().data[0])

        model.train(mode=was_training)

visualize_model_predictions(
    model_conv,
    img_path='/content/drive/MyDrive/CPE 313 - Advanced Machine
Learning and Deep Learning/Hands-on Activity 4.2 Transfer Learning on
PyTorch/hymenoptera_data/val/bees/72100438_73de9f17af.jpg'
)

plt.ioff()
plt.show()
```

Further Learning

If you would like to learn more about the applications of transfer learning, checkout the

- Quantized Transfer Learning for Computer Vision Tutorial.

Supplementary Activity

In a new notebook, perform the following:

1. Choose a pretrained model.
2. Finetune on your dataset from the previous activity.
3. Evaluate the performance of the previous model to this finetuned model.
4. Utilize the pretrained ConvNet model as fixed feature extractor.
5. Evaluate the performance of the previous model to this finetuned model.
6. Discuss the following:

- How did finetuning affect your performance?
- Which of the different situations for rule of thumb were applicable to you?